



ArxivLens

A Mobile Research Companion for Academic Paper Discovery and Citation Export

Team: Pavan Chauhan

Course: CS5035 — Topics in Functional Programming, Spring 2026

Instructor: Armando Beltran | CalStateLA

Project Overview: The Problem

01

Tab Overload

Students open dozens of arXiv tabs during a literature review and quickly lose track of what they read or why they saved it.

02

Manual Bookmarking

Papers land in notes apps or browser bookmarks — no structure, no abstracts, and no way to filter by topic or date.

03

BibTeX by Hand

Every citation must be rebuilt manually for LaTeX / Overleaf: copy author names, year, arXiv ID — error-prone and slow.

*There is no lightweight, mobile-first tool that covers the full loop:
search → read → save → export BibTeX — all offline, no sign-in.*

Project Overview: Who It's For

Target Users

- 1 CS students writing course papers with literature review sections
- 2 Graduate researchers tracking papers across multiple topics
- 3 Anyone using Overleaf / LaTeX who needs a portable .bib file

Why It Matters

A typical literature review involves:

10–30

papers per search session

Hours

spent manually building BibTeX

1 Tap

to export with ArxivLens

Your Approach: Tech Stack & API

Layer	Technology
UI	Jetpack Compose + Material 3
Navigation	Navigation Compose (NavController)
State	ViewModel + Kotlin Coroutines + StateFlow
Database	Room (SQLite) + KSP code generation
Networking	OkHttp (HTTP client)
Data Parsing	Android XmlPullParser — Atom feed
File Sharing	Android FileProvider + Share Intent

arXiv Atom API

Free · public · no auth required
Real-time metadata

Endpoint:
`https://export.arxiv.org/api/
query?search_query=all:...`

Returns:

- title, authors, abstract
- categories, arXiv ID
- PDF link, DOI

Your Approach: MVVM Architecture

UI Layer (Compose Screens)

Observes StateFlow — never reads DB or network directly



ViewModel Layer

Holds state, handles user actions, launches coroutines on viewModelScope



Repository (single data source)

Wraps Room DAOs, exposes Flow<T>, coordinates local & remote data



Room DAOs
(local SQLite)

arXiv API + XML Parser
(pure fn — no Android)

Key Principles

ViewModels never import
Android UI classes

Compose screens never
touch DB or network

BibTexExporter has zero
Android imports — pure

File I/O isolated in
Dispatchers.IO

Your Approach: Functional Programming Concepts

9 FP Concepts Demonstrated (7+ required by rubric)

Concept	Where in the codebase
Immutable data classes	Paper — pure value type, never mutated after construction
Data class: toString / copy / ==	Paper.toString(), ArxivXmlParser copy(), SearchViewModel == guard
Sealed UI states	SearchUiState, CollectionsUiState, CollectionDetailUiState
Lambda expressions	map { }, filter { } passed; SortOption.comparator() returned
Generics	topNBy<T,K>, splitBy<T>, frequencyMap<T> in FpUtils.kt
Generic constraints	<K : Comparable<K>> in topNBy — only orderable keys allowed
Control flow	when in parser + UI; for loop in frequencyMap; if/else throughout
Safe calls & Elvis	?.trim() ?: "" body?.string() totals[key] ?: 0
Kotlin Result type	search() returns Result<List<Paper>>, folded in ViewModel
Exception handling	try/catch in API service; .catch { } on Room Flows
Reactive streams (Flow)	Room Flow → combine → StateFlow; no manual state sync
Collection pipelines	splitBy + zip + map + frequencyMap + joinToString in BibTeX export

FP in Action: Sealed UI States

Idle

Search prompt shown.
App just opened, no results yet.

Loading

Spinner shown.
ArXiv API request is in flight.

Success

Paper list rendered.
LazyColumn of results.

Error

Error message shown.
Retry button offered.

Why sealed classes?

A single type covers every screen state. The when expression is exhaustive — the Kotlin compiler gives an error if any state is not handled. No boolean flags like isLoading or hasError that can contradict each other.

Used across all 3 screens

- SearchUiState — Idle / Loading / Success / Error
- CollectionsUiState — Loading / Success / Error
- CollectionDetailUiState — Loading / Success / Error

FP in Action: Higher-Order Functions & Lambda Expressions

How it works in ArxivLens

1 SortOption enum

Each of the 5 sort modes carries its own `Comparator<Paper>` as a function value inside the enum entry.

2 comparator() returns a lambda

The function returns a comparator — not a result. The UI just calls it; no switch/case logic anywhere in the screens.

3 Lambdas passed to collections

`map { }`, `filter { }`, `sortedWith { }` — lambdas are passed throughout the search and BibTeX pipelines.

4 Reactive combination

`displayState = combine(searchResults, sortOption)` — the sorted list is derived automatically whenever either changes.

5 Sort Modes (Search Screen)

Relevance

Default API order

Newest First

Sorted by published date, descending

Most Revised

Sorted by version number, descending

Peer Reviewed First

Papers with a DOI appear at top

Title A → Z

Alphabetical by title (case-insensitive)

FP in Action: Generics & Generic Constraints

Three reusable generic extension functions in `FpUtils.kt` — each used live in the BibTeX export pipeline:

`topNBy<T, K : Comparable<K>>`

What it does

Returns the top N items from a list, sorted by any property with natural ordering.

Constraint

`K : Comparable<K>` — the constraint means only types that support `< / >` can be used as the sort key (e.g. `Int`, `String`, date strings).

Used for

Could rank search results by relevance score or filter top N papers by year.

`frequencyMap<T>`

What it does

Counts how many times each item appears in a list. Uses a for-loop internally.

Constraint

Works for any type `T` — no constraint needed because `Map` keys just need equality.

Used for

Used in BibTeX export to detect duplicate cite keys (e.g. two "vaswani2017" entries) and add suffix letters: `vaswani2017a`, `vaswani2017b`.

`splitBy<T>`

What it does

Splits a list into two lists based on a predicate. A named, readable wrapper around Kotlin's `partition()`.

Constraint

Works for any type `T`. The predicate is a lambda: `(T) -> Boolean`.

Used for

Used in BibTeX export to put peer-reviewed papers (DOI present) before preprints in the output `.bib` file.

FP in Action: Immutable Data Classes

Paper — Pure Value Type

10 fields — all val, never mutated

Flows unchanged through the entire app:

arXiv API → XML Parser → ViewModel → UI

Every PaperCard in the search results renders from a different Paper object. Nothing in the UI writes back to the data.

`toString()`

Paper.kt

Custom readable format for logging and debugging.

Paper[2301.12345] "Attention Is All You Need" by Vaswani (2017)

Cite-key style — immediately tells you which paper it is.

`copy()`

ArxivXmlParser.kt

When the arXiv feed has no explicit PDF link, the parser derives it from the abstract URL using `copy()` — `rawPaper.copy(pdfUrl = arxivUrl.replace("/abs/", "/pdf/"))`. No mutation, no extra constructor.

`== structural equality`

SearchViewModel.kt

`selectPaper()` checks if `(_selectedPaper.value == paper)` before updating state. Data class equality compares all fields — tapping the same paper twice does nothing.

FP in Action: Pure BibTeX Export Pipeline

```
List<SavedPaper>
```

```
splitBy { doi.isNotBlank() }
```

```
map { buildCiteKey(it) }
```

```
frequencyMap() [for-loop]
```

```
zip + map { paperToBibTex() }
```

```
joinToString("\n\n") → .bib string
```

BibTexExporter — Pure

Zero Android imports · Zero I/O
Same input always → same .bib output

BibTexFileWriter — I/O Shell

Writes string to cache · Builds share intent
All disk I/O isolated here

Actual .bib output for one paper:

Paste directly into Overleaf — no edits needed

```
@misc{vaswani2017,  
  author      = {Ashish Vaswani and Noam Shazeer and Niki Parmar and Jakob Uszkoreit},  
  title       = {{Attention Is All You Need}},  
  year        = {2017},  
  eprint      = {1706.03762},  
  archivePrefix = {arXiv},  
  primaryClass = {cs.CL},  
  url         = {https://arxiv.org/abs/1706.03762},  
  doi         = {10.5555/3295222.3295349},  
}
```

Prototype: 4 Application Screens

1 Search

- Search field + sort dropdown (6 modes)
- LazyColumn of PaperCard results
- Save-to-collection bottom sheet
- New-collection dialog

2 Paper Detail

- Full title, authors, category chips
- Complete abstract
- "View on arXiv" and "Open PDF" buttons
- Save-to-collection from TopAppBar

3 My Collections

- LazyColumn with paper count badges
- Create collection (FAB + dialog)
- Delete with confirmation dialog
- Reactive paper counts via Flow

4 Collection Detail

- Papers ordered by save time
- Remove paper from collection
- Open on arXiv from list
- Export BibTeX (Extended FAB) + Share

Expected Impact: Before & After

Workflow Step	Before ArxivLens	With ArxivLens
Find papers	Multiple browser tabs	Single in-app search
Read abstract	Open each tab individually	In-app detail screen
Save paper	Browser bookmark or notes app	One tap → named collection
Build BibTeX	Manual copy-paste (error-prone)	One tap → .bib file
Send to Overleaf	Manual upload / copy-paste	Share sheet → Drive / email

Saves Time

Literature review citation work reduced from hours to minutes.

Reduces Errors

BibTeX generated from structured data — not typed by hand.

Works Offline

Saved collections and abstracts accessible without internet.

Expected Impact: Current Limitations

Internet required for search

The arXiv Atom API is remote. Search results cannot be fetched offline — only saved papers are available without a connection.

Local storage only

Collections live in on-device SQLite. No cross-device sync or cloud backup. Uninstalling the app loses all saved data.

All-fields search only

No dedicated author (au:), category (cat:), or date-range filters. Searches the full metadata text.

No offline abstract cache

Saved papers link back to arXiv for the full abstract. Abstract text is not stored locally in Room.

No unit tests yet

Pure functions (ArxivXmlParser, BibTexExporter, FpUtils) are ready for JUnit testing but no test files have been written.

Prototype Demo: Live Walkthrough

Search

1

Type "graph neural networks" → tap Search

Sort

2

Sort icon → "Newest first"

Paper Detail

3

Tap result → read abstract, note category chips

Save

4

Tap bookmark → create "GNN Papers" → save

Collections

5

Switch tab → tap Collections → open "GNN Papers"

Save Another

6

Back to Search → save a second paper

Export

7

Tap "Export BibTeX" FAB in Collection Detail

Share

8

Share sheet → show .bib file contents

Evaluation Plan: How We Test

Manual Testing — 10-step demo flow

Every feature tested end-to-end.

Edge cases: empty search, duplicate save, empty collection export, collection delete with papers.

Pure Functions — Ready for JUnit

ArxivXmlParser.parse(xml) — no Android needed

BibTexExporter.collectionToBibTex(papers)

FpUtils.topNBy / frequencyMap / splitBy



```
// Example test — ready to add:
@Test fun `BibTeX cite key dedup adds suffix
letters`() {
    val papers = listOf(
        paper("vaswani2017"),
        paper("vaswani2017")
    )
    val bib =
BibTexExporter.collectionToBibTex(papers)
    assertTrue(bib.contains("@misc{vaswani2017a,}")
    assertTrue(bib.contains("@misc{vaswani2017b,}")
}

// No Android emulator required.
// Pass plain Kotlin objects, assert the string.
```


What We Would Improve With More Time

Priority	Improvement
High	Add unit tests for ArxivXmlParser and BibTexExporter
High	Advanced search filters — author (au:), category (cat:), date range
Medium	Offline abstract caching — save full abstract text in Room
Medium	Reading status per paper (unread / reading / done)
Medium	Firebase Firestore sync for cross-device collections
Low	Collection-level search and sorting
Low	Personal notes per saved paper



Summary

ArxivLens is a native Android app built with Jetpack Compose, Room, and Kotlin coroutines that lets students and researchers search arXiv, save papers into named offline collections, and export BibTeX citations for direct use in LaTeX tools. The architecture follows MVVM with a clear data → repository → viewmodel → UI flow. Functional programming is applied throughout: immutable Paper value types flow from a pure XML parser through reactive StateFlow-backed UI state, and a pure BibTeX export pipeline — using `splitBy`, `frequencyMap`, `zip`, `map`, and `joinToString` — converts saved papers into a shareable .bib file without a single Android API call.